# Best Model Overview

## Reconstruir el ingreso de cada persona encuentra al 82 % de los hogares pobres

Universidad de los Andes

**0,78**

F1 final en Kaggle: **2.° lugar**

**0,782**

F1 en el 20 % apartado (holdout)

**35**

submissions, 7 algoritmos

### El modelo

1. Estimamos el ingreso de cada persona con lo que el test sí trae: ocupación, horas, cotización.
2. Lo sumamos por hogar y lo comparamos con la línea.
3. Un boosting de árboles (LightGBM) decide: pobre si $P \geq 0{,}36$.

### Lo que encontramos

1. Saber cuánto gana cada persona vale más que el algoritmo.
2. Nadie pasa de 0,78: un tercio de los pobres está casi en la línea.
3. Los excluidos están cerca de la línea.

Kaggle: tabla final (privada). LightGBM pesa 76 % de una mezcla con dos redes y un rediseño. Fuente: views/tablas/nivel7/etapa18_resumen.csv.




**Qué decir (1 min).** El gobierno quiere encontrar a los hogares pobres, pero la pobreza se define por el ingreso, y el ingreso es justo lo que no se puede preguntar bien. Nuestra respuesta fue reconstruirlo: estimamos cuánto gana cada persona con preguntas sencillas sobre su trabajo, lo sumamos por hogar y lo comparamos con la línea. Con eso quedamos segundos en Kaggle, con un F1 final de 0,78. Tres mensajes que vamos a sostener: primero, las buenas variables pesaron mucho más que el algoritmo; segundo, hay un techo cerca de 0,78 porque muchos pobres están casi en la línea; y tercero, los hogares que el modelo deja por fuera están cerca de la línea, no en la pobreza extrema.

**Pregunta del profesor: ¿por qué una mezcla y no un solo modelo? Respuesta:** La mezcla gana menos de una milésima frente al LightGBM solo, y esa ventaja no sobrevive a una prueba más estricta. La subimos porque cumplió la regla que fijamos antes, pero a una agencia le recomendaríamos el LightGBM solo.

# Ser pobre depende del ingreso, y el ingreso es justo lo que no podemos preguntar

Universidad de los Andes

Pobre: ingreso por persona bajo la línea de su ciudad. El test no trae ingresos.

- Preguntarlo es caro, la gente lo esconde y cambia mes a mes.
- “Nadie es pobre” acierta 80 %, pero no encuentra a ningún pobre: usamos F1.

**164.960**

hogares para entrenar

**1 de 5**

es pobre

**Hay que reconstruir el ingreso con lo que sí se ve.**

66.168 hogares para predecir. 79 variables solo existen en train, entre ellas todos los ingresos. Fuente: views/tablas/variables_ausentes_test_*.csv; views/tablas/desbalance_clases.csv.



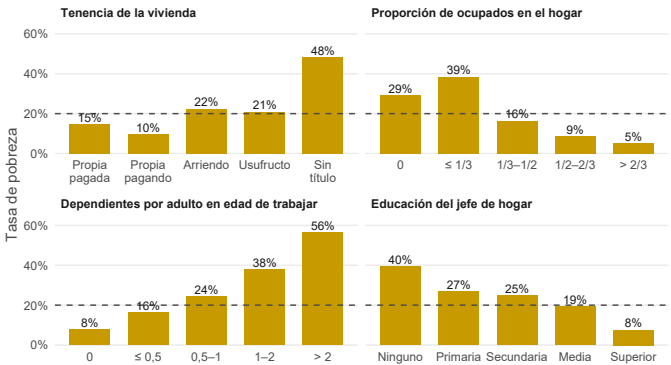

**Qué decir (1 min).** La regla de pobreza es mecánica: si el ingreso por persona del hogar está bajo la línea de su ciudad, el hogar es pobre. El problema es que el test no trae ningún ingreso, igual que pasaría en una encuesta corta del gobierno: preguntar el ingreso es caro, la gente lo subreporta y además cambia mes a mes. Además, solo uno de cada cinco hogares es pobre. Un modelo que diga que nadie es pobre acierta en el 80 % de los casos, pero no le sirve a nadie. Por eso usamos el F1, que castiga tanto dejar pobres por fuera como darle el subsidio a quien no lo necesita.

**Pregunta del profesor: ¿por qué no predecir el ingreso y aplicar la línea? Respuesta:** El taller pide clasificar, y eso hace el modelo final. El ingreso lo usamos solo para enseñarle a un modelo auxiliar, nunca como variable del hogar.

# Cuántos trabajan y cuántos dependen ya separan a pobres de no pobres

Universidad de los Andes

Tasa de pobreza en train (164.960 hogares). Línea punteada: promedio (20,0%)




**Ingreso por persona** = cuántos trabajan × cuánto gana cada uno ÷ cuántos comen.

- Más dependientes: pobreza de 8 % a 56 %.
- Jefe sin estudios: 40 %; con universidad: 8 %.
- Sin ocupados (29 %) es menos pobre que un tercio ocupado (39 %): pensiones.

## Ninguna variable sola basta: la mejor da un F1 de 0,47.

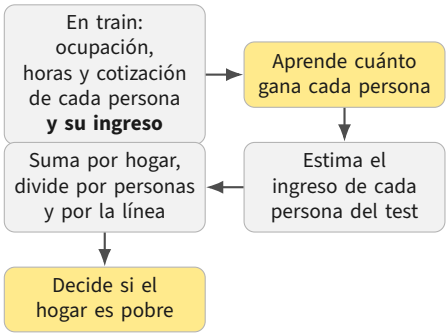
Tasa de pobreza en los 164.960 hogares de train. Fuente: scripts/13_figuras_presentacion.R; views/tablas/leaderboard_interno.csv.




**Qué decir (1 min).** Si no vemos el ingreso, partimos de cómo se forma: cuántas personas trabajan, cuánto gana cada una y entre cuántas se reparte. Los datos confirman cada pieza. Cuando hay más dependientes por adulto, la pobreza pasa de 8 a 56 %. La educación del jefe también separa mucho. Y la vivienda importa: quien paga arriendo es más pobre que quien tiene casa propia pagada. Hay una sorpresa: los hogares donde nadie trabaja no son los más pobres, porque muchos son pensionados. Por eso la relación no es lineal, y por eso ninguna variable sola basta.

**Pregunta del profesor: ¿basta con estas variables en un Logit? Respuesta:** No: un Logit con 5 variables de la teoría da F1 de 0,46. Sube a 0,72 cuando construimos 48 variables a partir de cada persona del hogar.

# Si no vemos el ingreso, lo estimamos persona por persona

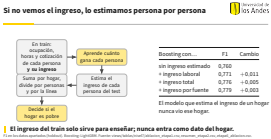



| Boosting con… | F1 | Cambio |
|---|---|---|
| sin ingreso estimado | 0,760 | |
| + ingreso laboral | 0,771 | +0,011 |
| + ingreso total | 0,776 | +0,005 |
| + ingreso por fuente | 0,779 | +0,003 |

El modelo que estima el ingreso de un hogar nunca vio ese hogar.

**El ingreso del train solo sirve para enseñar; nunca entra como dato del hogar.**

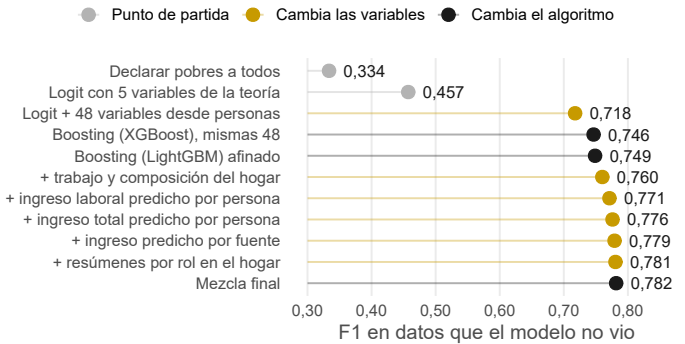
F1 en los datos apartados (holdout). Boosting: LightGBM. Fuente: views/tablas/nivel7/ablacion_etapa1.csv, resumen_etapa2.csv, etapa6_ablacion.csv.




**Qué decir (1 min).** Esta es la idea central. En train sí sabemos cuánto gana cada persona. Entonces entrenamos un modelo que aprende a predecir ese ingreso usando solo lo que el test también trae: si trabaja, cuántas horas, si cotiza a pensión, su oficio. En el test aplicamos ese modelo a cada persona, sumamos por hogar, dividimos por el número de personas y por la línea. Ese número se vuelve la variable más poderosa del clasificador. Solo el ingreso laboral estimado sube el F1 más de un punto.

**Pregunta del profesor: ¿eso no es usar la respuesta para predecir? Respuesta:** No, porque el modelo que estima el ingreso de un hogar nunca vio ese hogar, ni en el entrenamiento ni en la validación. Lo verificamos con pruebas automáticas y con una prueba de que train y test se parecen (apéndice).

# Saber cuánto gana cada persona vale más que el algoritmo



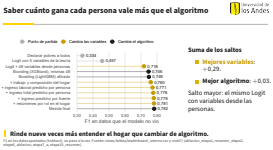

**Suma de los saltos**

- **Mejores variables**: $+0{,}29$.
- **Mejor algoritmo**: $+0{,}03$.

Salto mayor: el mismo Logit con variables desde las personas.

**Rinde nueve veces más entender el hogar que cambiar de algoritmo.**

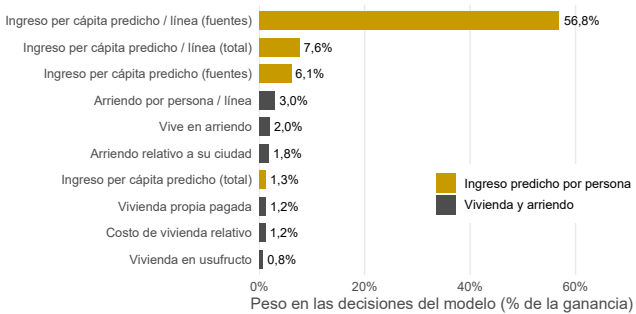
F1 en los datos apartados (holdout), un paso a la vez. Fuente: views/tablas/leaderboard_interno.csv y nivel7/ (ablacion_etapa1, resumen_etapa2, etapa6_ablacion, etapa7_a, etapa18_resumen).




**Qué decir (1 min).** Esta escalera resume todo el proceso. Cada punto es un paso, medido siempre en los mismos datos apartados. Los puntos amarillos cambian las variables y dejan fijo el algoritmo; los negros cambian el algoritmo y dejan fijas las variables. Si sumamos, las variables aportaron 0,29 de F1 y los algoritmos 0,03. El salto más grande no tuvo nada de sofisticado: el mismo Logit, pero con variables construidas a partir de las personas del hogar.

**Pregunta del profesor: ¿no depende del orden en que hicieron los pasos? Respuesta:** Sí, la suma es secuencial. Pero el contraste se sostiene en cualquier orden: con LightGBM fijo las variables nuevas suman 0,032, más que toda la distancia entre el Logit y el boosting con las mismas variables.

# El modelo decide con el ingreso estimado y con el arriendo




| Grupo | Variables | Peso |
|---|---|---|
| Ingreso estimado | 17 | 76 % |
| Vivienda y arriendo | 15 | 11 % |
| Resúmenes por rol | 1.026 | 9 % |
| Resto | 266 | 4 % |

El arriendo revela cuánto puede pagar el hogar; la casa propia pagada es un ingreso escondido.

## El modelo reconstruye el ingreso y lo corrige por el costo de vivir.

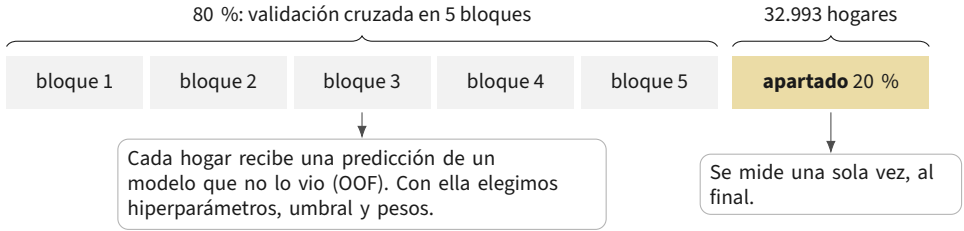
Peso: cuánto reduce el error cada variable en los árboles del LightGBM (76 % de la mezcla). Fuente: views/tablas/nivel7/etapa7_a_importancia.csv, etapa15_ebm_importancia.csv.




**Qué decir (1 min).** ¿Qué aprendió el modelo? Tres cuartas partes de sus decisiones vienen de 17 variables de ingreso estimado. Le sigue la vivienda: cuánto paga de arriendo comparado con la línea y si la casa es propia. Tiene sentido económico: el arriendo muestra capacidad de pago, y vivir en casa propia pagada es como recibir un ingreso que nadie reporta. Las más de mil variables de resumen por rol en el hogar pesan poco.

**Pregunta del profesor: esta medida de importancia favorece variables continuas. Respuesta:** Es cierto, por eso la contrastamos con un boosting interpretable (EBM) que mide el aporte de otra forma. Ahí también mandan el ingreso estimado y el arriendo por persona.

# Todo se decidió con datos que el modelo no vio




**Por qué gana el boosting:** suma 1.852 árboles pequeños y cada uno corrige los errores de los anteriores. Así capta cortes y combinaciones que un Logit no ve.

**Repetir todo con otras particiones da el mismo F1: 0,775.**

Fuente: views/tablas/resumen_particion.csv; views/tablas/nivel7/etapa16_fase1_resumen.csv (3 particiones nuevas, 15 evaluaciones).




**Qué decir (1 min).** Así entrenamos. Al principio separamos un 20 % de los hogares y no lo tocamos. Con el otro 80 % hacemos validación cruzada: se entrena con cuatro bloques y se predice el quinto, rotando. Así, cada hogar recibe una predicción de un modelo que no lo vio, y con esas predicciones elegimos todo: hiperparámetros, umbral y pesos de la mezcla. ¿Por qué gana el boosting? Porque va sumando árboles pequeños que corrigen los errores que quedan, y esos errores están justo en los hogares difíciles, cerca de la línea. Al final repetimos todo con tres particiones nuevas y el resultado no cambió.

**Pregunta del profesor: miraron muchas veces los datos apartados, ¿no se sobreajustaron? Respuesta:** Los miramos, pero nunca para elegir parámetros, y por eso la comparación final la hicimos con particiones nuevas. Además, Kaggle fue la prueba externa: 0,78 en la tabla final, igual que en nuestros datos apartados.

# Afinar y balancear las clases casi no mueve el F1

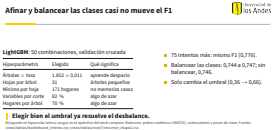
**LightGBM**: 50 combinaciones, validación cruzada

| Hiperparámetro | Elegido | Qué significa |
|---|---|---|
| Árboles × tasa | 1.852 × 0,011 | aprende despacio |
| Hojas por árbol | 31 | árboles pequeños |
| Mínimo por hoja | 171 hogares | no memoriza casos |
| Variables por corte | 82 % | algo de azar |
| Hogares por árbol | 70 % | algo de azar |

- 75 intentos más: mismo F1 (0,776).
- Balancear las clases: 0,744 a 0,747; sin balancear, 0,746.
- Solo cambia el umbral (0,36 → 0,66).

**Elegir bien el umbral ya resuelve el desbalance.**

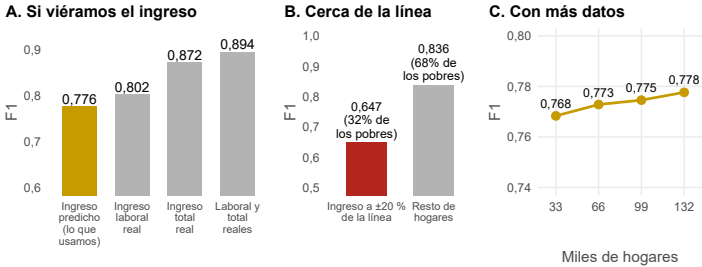
Búsqueda en hipercubo latino; rangos en el apéndice del deck compare. Balanceo: pobres sintéticos (SMOTE), submuestreo y pesos de clase. Fuente: views/tablas/leaderboard_interno.csv; views/tablas/nivel7/resumen_etapa2.csv.




**Qué decir (1 min).** Buscamos los hiperparámetros con 50 combinaciones, eligiendo con validación cruzada. La configuración ganadora es prudente: muchos árboles pequeños, aprendizaje lento y hojas con al menos 171 hogares, para que no memorice casos raros. Buscar 75 combinaciones más no mejoró nada. Con el desbalance pasa lo mismo: crear pobres sintéticos, quitar no pobres o darles más peso a los pobres deja el F1 igual. Lo único que cambia es el umbral, porque balancear solo infla las probabilidades.

**Pregunta del profesor: ¿por qué no afinar con los datos apartados? Respuesta:** Porque dejarían de ser una medida limpia. Todo el afinamiento usó solo la validación cruzada del 80 %.

# Ningún modelo pasa de 0,78: un tercio de los pobres está casi en la línea



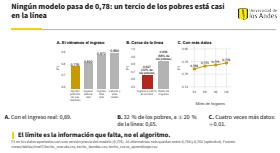

**A.** Con el ingreso real: 0,89.

**B.** 32 % de los pobres, a $\pm$ 20 % de la línea: 0,65.

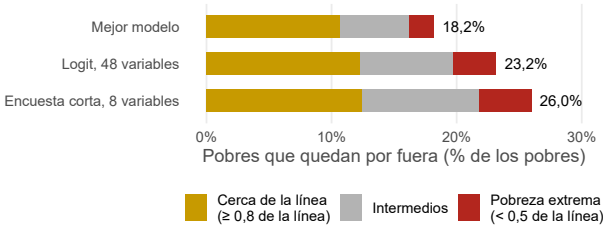
**C.** Cuatro veces más datos: $+$0,01.

## El límite es la información que falta, no el algoritmo.

F1 en los datos apartados con una versión previa del modelo (0,776). 16 alternativas más quedan entre 0,750 y 0,782 (apéndice). Fuente: views/tablas/nivel7/techo_oraculo.csv, techo_bandas.csv, techo_curva_aprendizaje.csv.


Ningún modelo pasa de 0,78: un tercio de los pobres está casi en la línea
Universidad de los Andes
A. El mínimo: el ingreso
B. Cerca de la línea
C. Con más datos
F1
0.65
0.78
0.87
0.89
0.65
0.65
0.65

# Los pobres que se nos escapan están justo en la línea




- **Subcobertura** (pobres por fuera): 18 %; seis de cada diez, cerca de la línea.
- **Filtración** (reciben sin necesitarlo): 25 %, más cerca de la línea que el no pobre típico.

**Pesando cada error por su distancia a la línea: subcobertura 11 %, filtración 1 %.**

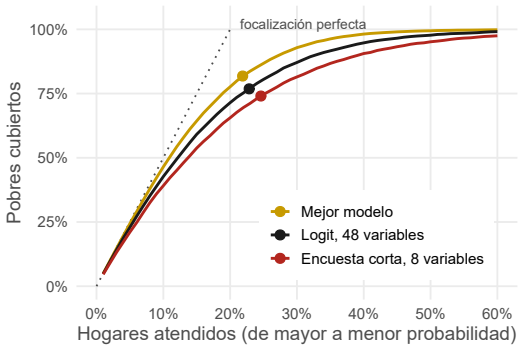
Datos apartados (holdout), umbral 0,36. Cerca de la línea: ingreso $\geq$ 0,8 veces la línea. Pobreza extrema: $<$ 0,5. Mezcla sin el rediseño (95 % del peso final).
Fuente: views/tablas/nivel7/etapa15_pesos_bienestar.csv, etapa15_errores_subgrupo.csv.




**Qué decir (1 min).** Cada error tiene un costo humano, pero no todos cuestan lo mismo. El modelo deja por fuera al 18 % de los pobres, y seis de cada diez de ellos tienen un ingreso de al menos 80 % de la línea. El rojo, la pobreza extrema, es la parte más pequeña. Del otro lado, uno de cada cuatro atendidos no es pobre, pero están mucho más cerca de la línea que el no pobre típico: su distancia media a la línea es 0,51, frente a 2,83 del no pobre promedio. Si contamos cada error según qué tan lejos está de la línea, la subcobertura baja a 11 % y la filtración casi desaparece.

**Pregunta del profesor: ¿el modelo discrimina a algún grupo? Respuesta:** Hay más subcobertura en hogares de una o dos personas (34 % y 28 %) y en Manizales y Pereira, porque ahí el ingreso depende de una sola persona. La agencia podría usar un umbral más bajo para esos grupos.

# El umbral es una decisión de política, no estadística




El F1 supone que ambos errores cuestan igual.

- Umbral de 0,38 a 0,21: subcobertura de 19 % a 10 %.
- Filtración de 27 % a 36 %; F1 de 0,77 a 0,75.

**Si dejar a un pobre por fuera cuesta más, conviene bajar el umbral.**

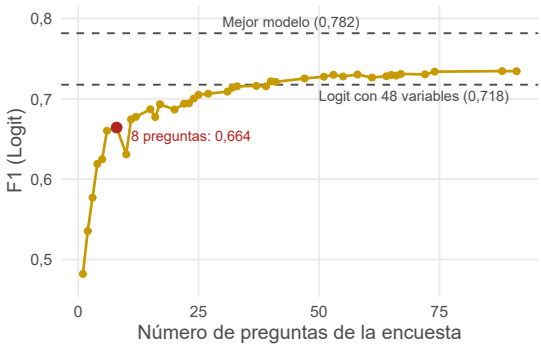
Curva y puntos en los datos apartados (holdout); umbrales calculados con una versión previa del modelo. Fuente: views/tablas/nivel7/etapa15_curva_cobertura.csv; views/tablas/escenarios_politica_n7.csv.




**Qué decir (1 min).** El umbral que maximiza el F1 trata igual los dos errores. Pero un gobierno puede pensar que dejar por fuera a una familia pobre es peor que darle el subsidio a una familia que está apenas encima de la línea. Si es así, puede bajar el umbral: la subcobertura se reduce a la mitad, de 19 a 10 %, a cambio de unos diez puntos más de filtración. Y como los filtrados están relativamente cerca de la línea, ese costo es moderado. La curva muestra que atendiendo al 30 % de los hogares ya cubrimos al 93 % de los pobres.

**Pregunta del profesor: ¿qué umbral recomiendan? Respuesta:** Depende de cuánto más cuesta excluir que filtrar, y esa es una decisión normativa. Con costos iguales, el del F1; si excluir cuesta el doble, uno más bajo.

# Una encuesta corta ahorra plata, pero deja por fuera a más pobres extremos




Nuestro modelo necesita el módulo de trabajo de cada persona: largo y caro.

- 8 preguntas: F1 de 0,66.
- Pobres por fuera: 26 % (mejor modelo: 18 %).
- De ellos, en pobreza extrema: 16 % (11 %).

**Lo que se ahorra en la encuesta lo pagan los más pobres.**

Logit, datos apartados (holdout). Las 8 preguntas: ocupados, horas, cotizantes, quién quiere trabajar más, arriendo pagado y recibido, dormitorios; elegidas con un algoritmo genético. Fuente: views/tablas/nivel7/frontera_encuesta_corta.csv, etapa15_pesos_bienestar.csv.




**Qué decir (1 min).** ¿Se puede usar esto con una encuesta corta y barata? Buscamos el mejor grupo de preguntas para cada tamaño de encuesta. Con 8 preguntas (cuántos trabajan, cuántas horas, cuántos cotizan, el arriendo y los dormitorios) el F1 baja de 0,78 a 0,66. Lo más grave no es el número, es a quién excluye: la encuesta corta deja por fuera a más pobres, y más de ellos en pobreza extrema. Ahorrar en la encuesta se paga con los hogares más vulnerables.

**Pregunta del profesor: ¿no pueden estimar el ingreso por persona también en la encuesta corta? Respuesta:** No sin el módulo de trabajo, porque ese modelo se alimenta de la ocupación, las horas y la cotización de cada persona. Por eso la encuesta corta se evaluó sin él.

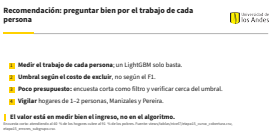
# Recomendación: preguntar bien por el trabajo de cada persona

Universidad de los Andes

1. **Medir el trabajo de cada persona**; un LightGBM solo basta.
2. **Umbral según el costo de excluir**, no según el F1.
3. **Poco presupuesto:** encuesta corta como filtro y verificar cerca del umbral.
4. **Vigilar** hogares de 1–2 personas, Manizales y Pereira.

**El valor está en medir bien el ingreso, no en el algoritmo.**

Encuesta corta: atendiendo al 40 % de los hogares cubre al 91 % de los pobres. Fuente: views/tablas/nivel7/etapa15_curva_cobertura.csv, etapa15_errores_subgrupo.csv.




**Qué decir (1 min).** Cerramos con lo que le diríamos a una agencia. Primero, la inversión que más rinde es preguntar bien por el trabajo de cada persona; con eso, un LightGBM solo basta. Segundo, el umbral es una decisión de política: si excluir a un pobre es grave, hay que bajarlo. Tercero, si solo alcanza para una encuesta corta, usarla como filtro amplio y verificar en campo a los hogares cercanos al umbral, que es donde el modelo duda. Y cuarto, vigilar a los grupos con más subcobertura. No esperen mucho de más modelos: el límite es de información.

**Pregunta del profesor: ¿se pueden verificar en campo las variables clave? Respuesta:** En parte: la vivienda y el arriendo se ven en una visita, pero las horas y la ocupación son autorreportadas. Por eso proponemos verificar solo a los hogares cerca del umbral, donde un error cambia la decisión.

# Glosario (1/2): cómo medimos los errores

| | |
|---|---|
| **Precisión** | De los hogares que el programa atiende, qué parte es pobre. |
| **Filtración** | 1−precisión: hogares no pobres que reciben el subsidio sin necesitarlo. |
| **Recall** | De los hogares pobres, qué parte atiende el programa. |
| **Subcobertura** | 1−recall: hogares pobres que se quedan por fuera. |
| **F1** | Promedio (armónico) de precisión y recall: solo es alto si ambos lo son. |
| **Umbral** | Probabilidad desde la que un hogar se declara pobre. Bajarlo cubre más pobres a cambio de más filtración. |

**Glosario (1/2): cómo medimos los errores**

Universidad de los Andes

| | |
|---|---|
| **Precisión** | De los hogares que el programa atiende, qué parte es pobre. |
| **Filtración** | 1 – precisión: hogares no pobres que reciben el subsidio sin necesitarlo. |
| **Recall** | De los hogares pobres, qué parte atiende el programa. |
| **Subcobertura** | 1 – recall: hogares pobres que se quedan por fuera. |
| **F1** | Promedio (armónico) de precisión y recall: solo es alto si ambos lo son. |
| **Umbral** | Probabilidad desde la que un hogar se declara pobre. Bajarlo cubre más pobres a cambio de más filtración. |

Glosario de apoyo; se muestra solo si lo preguntan.

# Glosario (2/2): cómo validamos

Universidad de los Andes

| | |
|---|---|
| **Datos apartados (holdout)** | 20 % del train (32.993 hogares) separado al inicio; solo sirve para medir, nunca para decidir. |
| **Validación cruzada** | El 80 % restante se parte en 5 bloques: se entrena con 4 y se evalúa en el quinto, rotando. |
| **Predicción sobre datos no vistos (OOF)** | La predicción de cada hogar hecha por un modelo que no lo vio. Con ella elegimos umbral, pesos e hiperparámetros. |
| **Modelo de ingreso por persona** | Estima cuánto gana cada persona con lo que el test sí trae; sumado al hogar, entra como variable. El que alimenta a un hogar nunca vio ese hogar. |
| **Diferencia más pequeña que podemos medir** | Con 32.993 hogares, mejoras menores a $\approx 0{,}0025$ de F1 no se distinguen del ruido. |

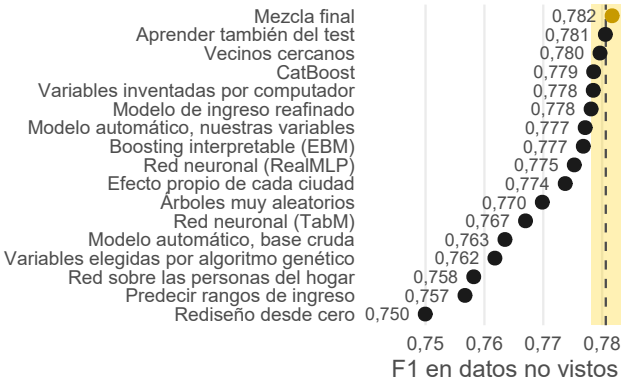
## Glosario (2/2): cómo validamos

Universidad de los Andes

| | |
|---|---|
| **Datos apartados (holdout)** | 20 % del train (32.993 hogares) separado al inicio; solo sirve para medir, nunca para decidir. |
| **Validación cruzada** | El 80 % restante se parte en 5 bloques: se entrena con 4 y se evalúa en el quinto, rotando. |
| **Predicción sobre datos no vistos (OOF)** | La predicción de cada hogar hecha por un modelo que no lo vio. Con ella elegimos umbral, pesos e hiperparámetros. |
| **Modelo de ingreso por persona** | Estima cuánto gana cada persona con lo que el test sí trae; sumado al hogar, entra como variable. El que alimenta a un hogar nunca vio ese hogar. |
| **Diferencia más pequeña que podemos medir** | Con 32.993 hogares, mejoras menores a ≈ 0.0025 de F1 no se distinguen del ruido. |

Glosario de apoyo; se muestra solo si lo preguntan.

# Apéndice: dieciséis alternativas más chocan con el mismo techo




Banda amarilla: la diferencia más pequeña que podemos medir ($\pm$0,0025) alrededor del LightGBM base (línea punteada).

- Ninguna alternativa supera al LightGBM por fuera de la banda.
- La mezcla final gana $+$0,0007 en validación repetida.

F1 en los datos apartados (holdout). Fuente: views/tablas/nivel7/ (etapa7_b, etapa8 a etapa18); banda de etapa16_fase1_resumen.csv.




Respaldo para “¿probaron otra cosa?”. Redes neuronales, modelos automáticos, CatBoost, efectos por ciudad, vecinos cercanos, variables creadas por computador, aprender del test sin etiqueta y un rediseño desde cero: todos quedan dentro o por debajo de la banda de ruido.

# Apéndice: no hay fuga de información y train se parece al test

Universidad de los Andes

## Construcción

- 0 personas sin hogar; 0 hogares sin jefe o con varios jefes.
- Pobre $=$ (ingreso por persona $<$ línea) en los 164.960 hogares.
- Ningún hogar está a la vez en train y en test.

## Prueba de que train y test se parecen

- Un clasificador no logra distinguirlos (AUC 0,501) con las variables observadas.
- Con el ingreso estimado sube a 0,713, porque en test se calcula con el modelo completo. Corregirlo no cambia el F1.

Validación adversarial. Fuente: views/tablas/nivel7/forense_resumen.csv, forense_adversarial.csv, forense_experimentos.csv.

## Apéndice: no hay fuga de información y train se parece al test

Universidad de los Andes

**Construcción**

- 0 personas sin hogar; 0 hogares sin jefe o con varios jefes.
- Pobre = (ingreso por persona < línea) en los 164.960 hogares.
- Ningún hogar está a la vez en train y en test.

**Prueba de que train y test se parecen**

- Un clasificador no logra distinguirlos (AUC 0,501) con las variables observadas.
- Con el ingreso estimado sube a 0,713, porque en test se calcula con el modelo completo. Corregirlo no cambia el F1.

Validación adversarial. Fuente: views/tablas/cmd73/exercise_resumen.csv, /exercise_adversarial.csv, /exercise_experimentos.csv.

Respaldo para preguntas sobre fuga o sobre diferencias entre train y test.

# Apéndice: en validación repetida, ninguna alternativa mejora de forma medible

| Candidato | F1 validación repetida (EE) | Diferencia vs. LightGBM [IC 95 %] | F1 apartado [IC 95 %] |
|---|---|---|---|
| LightGBM base | 0,7752 (0,0008) | | 0,7806 [0,7731; 0,7885] |
| + redes (mezcla sin rediseño) | 0,7758 (0,0008) | +0,0006 [−0,0001; +0,0013] | 0,7820 [0,7743; 0,7897] |
| **+ redes + rediseño (final)** | **0,7759 (0,0008)** | +0,0007 [+0,0002; +0,0012] | 0,7817 [0,7742; 0,7894] |
| CatBoost | 0,7722 (0,0007) | −0,0030 [−0,0042; −0,0018] | 0,7786 [0,7707; 0,7864] |
| Base con códigos corregidos | 0,7754 (0,0008) | +0,0002 [−0,0002; +0,0005] | 0,7797 [0,7720; 0,7875] |
| Rediseño desde cero | 0,7478 (0,0008) | −0,0274 [−0,0289; −0,0260] | 0,7499 [0,7420; 0,7581] |

15 evaluaciones (3 particiones nuevas $\times$ 5 bloques), IC pareados ingenuos. Con la corrección de Nadeau y Bengio por dependencia entre bloques, el IC del modelo final es $[-0{,}0004; +0{,}0017]$. Fuente: views/tablas/nivel7/etapa16_fase1_resumen.csv, etapa18_resumen.csv.

**Apéndice: en validación repetida, ninguna alternativa mejora de forma medible**

Universidad de los Andes

| Candidato | F1: validación repetida (EE) | Diferencia vs. LightGBM [IC 95 %] | F1 apartado [IC 95 %] |
|---|---|---|---|
| LightGBM tal cual | 0,7732 (0,0008) | | 0,7606 [0,7731; 0,7861] |
| + redes (mezcla sin rediseño) | 0,7736 (0,0008) | +0,0006 [−0,0001; +0,0013] | 0,7620 [0,7745; 0,7897] |
| **+ redes + rediseño (final)** | **0,7739 (0,0008)** | **+0,0007 [+0,0002; +0,0012]** | 0,7617 [0,7742; 0,7894] |
| Rediseño solo | 0,7734 (0,0008) | +0,0002 [−0,0002; +0,0006] | 0,7607 [0,7732; 0,7883] |
| Base con código ecoregión | 0,7734 (0,0008) | +0,0002 [−0,0002; +0,0005] | 0,7747 [0,7720; 0,7875] |
| Rediseño desde cero | 0,7478 (0,0008) | −0,0274 [−0,0289; −0,0260] | 0,7499 [0,7430; 0,7581] |

El modelo final es [−0,0004; +0,0017]. Fuente: corrida habilitadora07_base2_recursos.ipynb, celda 15, recursos.csv.

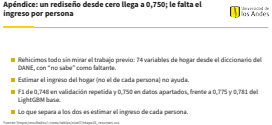
Respaldo: la única diferencia cuyo IC ingenuo excluye cero es la del modelo final, por milésimas, y deja de excluirlo con la corrección.

# Apéndice: un rediseño desde cero llega a 0,750; le falta el ingreso por persona

Universidad de los Andes

- Rehicimos todo sin mirar el trabajo previo: 74 variables de hogar desde el diccionario del DANE, con “no sabe” como faltante.
- Estimar el ingreso del hogar (no el de cada persona) no ayuda.
- F1 de 0,748 en validación repetida y 0,750 en datos apartados, frente a 0,775 y 0,781 del LightGBM base.
- Lo que separa a los dos es estimar el ingreso de cada persona.

Fuente: limpio/resultados/; views/tablas/nivel7/etapa18_resumen.csv.




Respaldo: el rediseño confirma que estimar el ingreso de cada persona es la pieza clave.